

```
In [1]: # Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.stattools import durbin_watson
from scipy import stats
from sklearn.ensemble import RandomForestRegressor
sns.set(style="whitegrid") # standardize plots
```

```
In [2]: #Loading the data
df=pd.read_csv("student-por.csv",sep=';')
```

```
In [3]: #DATA PREPROCESSING
#Checking first 10 rows
df.head(10)
```

```
Out[3]:
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4
3	GP	F	15	U	GT3	T	4	2	health	services	...	3
4	GP	F	16	U	GT3	T	3	3	other	other	...	4
5	GP	M	16	U	LE3	T	4	3	services	other	...	5
6	GP	M	16	U	LE3	T	2	2	other	other	...	4
7	GP	F	17	U	GT3	A	4	4	other	teacher	...	4
8	GP	M	15	U	LE3	A	3	2	services	other	...	4
9	GP	M	15	U	GT3	T	3	4	other	other	...	5

10 rows × 33 columns



```
In [4]: #Checking no of rows and columns
df.shape
```

```
Out[4]: (649, 33)
```

```
In [6]: #Checking info
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 649 entries, 0 to 648
Data columns (total 33 columns):
#   Column          Non-Null Count  Dtype
---  -
0   school          649 non-null    object
1   sex              649 non-null    object
2   age              649 non-null    int64
3   address          649 non-null    object
4   famsize          649 non-null    object
5   Pstatus          649 non-null    object
6   Medu              649 non-null    int64
7   Fedu              649 non-null    int64
8   Mjob              649 non-null    object
9   Fjob              649 non-null    object
10  reason           649 non-null    object
11  guardian         649 non-null    object
12  traveltime       649 non-null    int64
13  studytime        649 non-null    int64
14  failures         649 non-null    int64
15  schoolsup         649 non-null    object
16  famsup           649 non-null    object
17  paid              649 non-null    object
18  activities        649 non-null    object
19  nursery          649 non-null    object
20  higher           649 non-null    object
21  internet         649 non-null    object
22  romantic         649 non-null    object
23  famrel           649 non-null    int64
24  freetime         649 non-null    int64
25  goout            649 non-null    int64
26  Dalc              649 non-null    int64
27  Walc              649 non-null    int64
28  health           649 non-null    int64
29  absences         649 non-null    int64
30  G1                649 non-null    int64
31  G2                649 non-null    int64
32  G3                649 non-null    int64
dtypes: int64(16), object(17)
memory usage: 167.4+ KB

```

```

In [7]: #Checking for missing values
df.isnull().sum()

```

```
Out[7]: school      0
sex              0
age              0
address          0
famsize          0
Pstatus          0
Medu             0
Fedu             0
Mjob             0
Fjob            0
reason           0
guardian         0
traveltime       0
studytime        0
failures         0
schoolsup        0
famsup           0
paid             0
activities       0
nursery          0
higher           0
internet         0
romantic         0
famrel           0
freetime         0
goout            0
Dalc             0
Walc             0
health           0
absences         0
G1              0
G2              0
G3              0
dtype: int64
```

```
In [8]: df.describe()
```

Out[8]:

	age	Medu	Fedu	traveltime	studytime	failures	famrel
count	649.000000	649.000000	649.000000	649.000000	649.000000	649.000000	649.000000
mean	16.744222	2.514638	2.306626	1.568567	1.930663	0.221880	3.930663
std	1.218138	1.134552	1.099931	0.748660	0.829510	0.593235	0.955717
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000	1.000000
25%	16.000000	2.000000	1.000000	1.000000	1.000000	0.000000	4.000000
50%	17.000000	2.000000	2.000000	1.000000	2.000000	0.000000	4.000000
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000	5.000000
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000	5.000000



```
In [10]: #Checking for duplicates
df.duplicated().any()
```

```
Out[10]: np.False_
```

```
In [11]: #Data analysis and visualization
df["school"].value_counts()
```

```
Out[11]: school
GP      423
MS      226
Name: count, dtype: int64
```

```
In [12]: df["sex"].value_counts()
```

```
Out[12]: sex
F      383
M      266
Name: count, dtype: int64
```

```
In [14]: df["famsize"].value_counts()
```

```
Out[14]: famsize
GT3     457
LE3     192
Name: count, dtype: int64
```

```
In [16]: import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 4, figsize=(12, 8))

# Gender pie chart
ax[0].pie(df["sex"].value_counts(),
          labels=df["sex"].value_counts().index,
          colors=['#e01bbf', '#2a77db'],
          autopct='%1.1f%%')
ax[0].set_title('Gender')

# School pie chart
ax[1].pie(df["school"].value_counts(),
          labels=df["school"].value_counts().index,
          autopct='%1.1f%%')
ax[1].set_title('School')

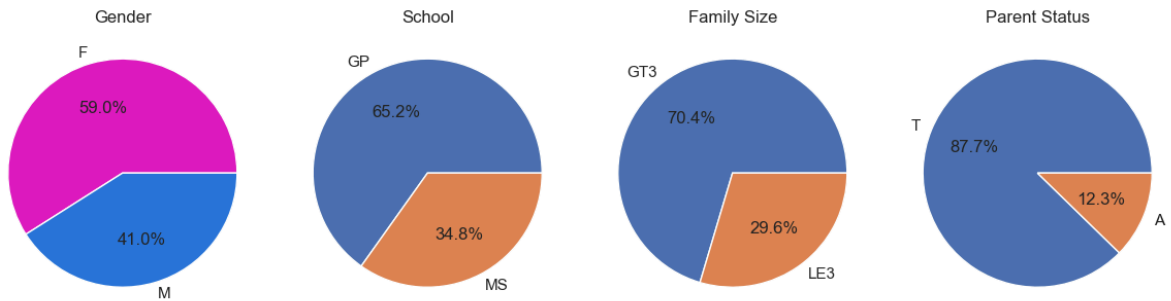
# Family Size pie chart
ax[2].pie(df["famsize"].value_counts(),
          labels=df["famsize"].value_counts().index,
          autopct='%1.1f%%')
ax[2].set_title('Family Size')

# Parent Status pie chart
ax[3].pie(df["Pstatus"].value_counts(),
          labels=df["Pstatus"].value_counts().index,
          autopct='%1.1f%%')
ax[3].set_title('Parent Status')
```

```
plt.tight_layout()

# Save the entire figure as an image
plt.savefig("pie_charts.png") # Can also use .jpg, .pdf, etc.

plt.show()
```



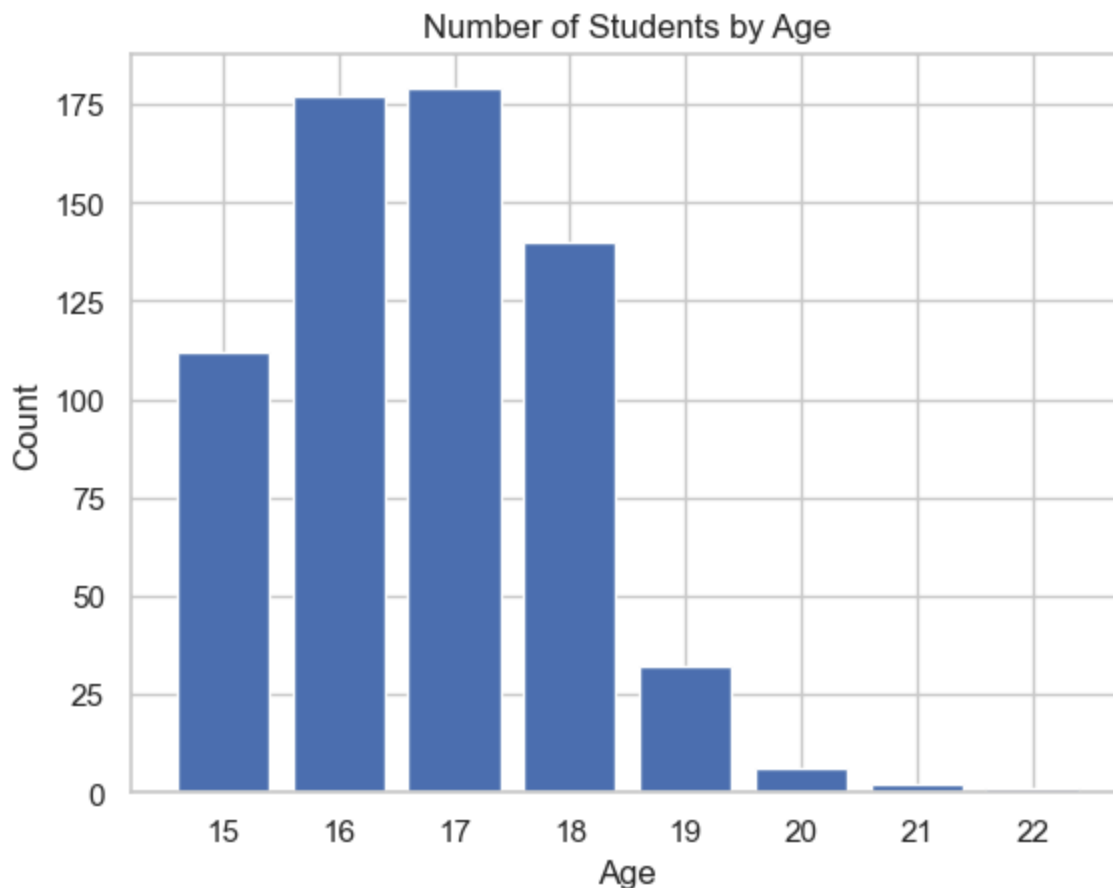
```
In [16]: df["age"].value_counts()
```

```
Out[16]: age
17      179
16      177
18      140
15      112
19       32
20        6
21         2
22         1
Name: count, dtype: int64
```

```
In [19]: # Plotting the bar chart
plt.bar(df["age"].value_counts().index, df["age"].value_counts().values)
plt.title("Number of Students by Age")
plt.xlabel("Age")
plt.ylabel("Count")

# Save the figure as a PNG file
plt.savefig("students_by_age.png") # You can also use .jpg, .pdf, etc.

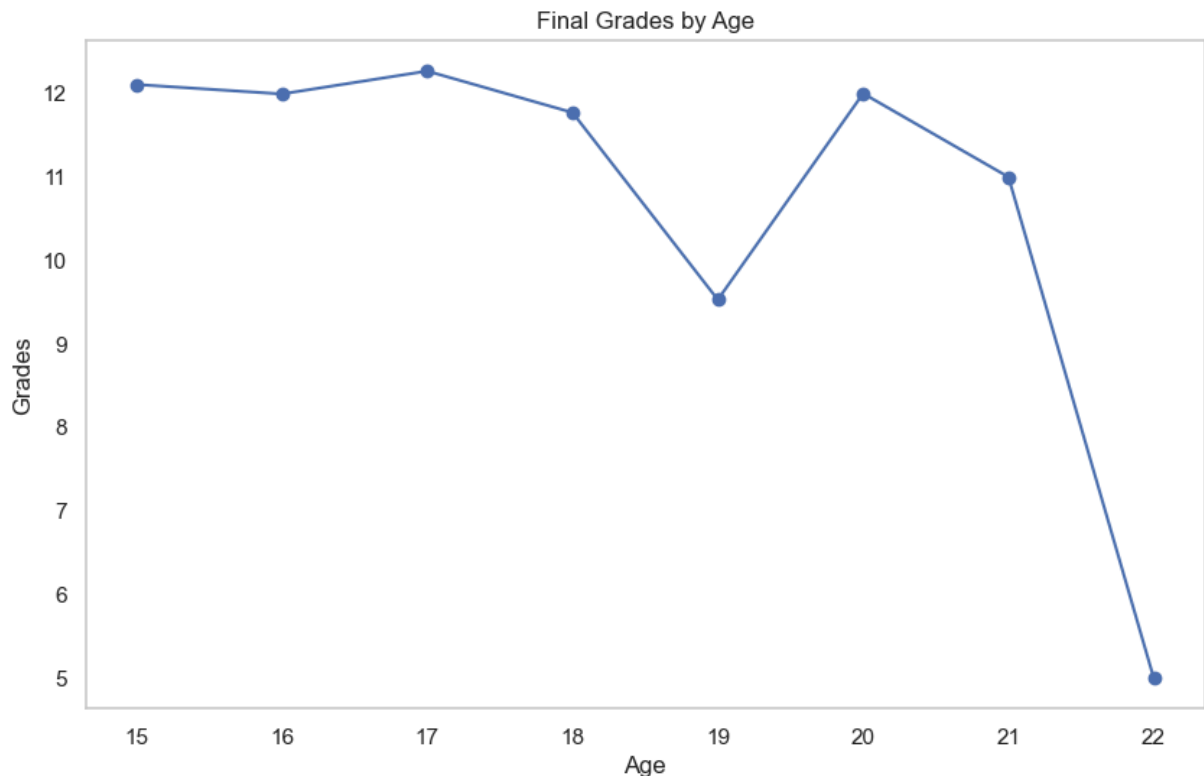
# Display the plot
plt.show()
```



```
In [20]: age_gp = df.groupby('age')['G3'].mean()  
age_gp
```

```
Out[20]: age  
15    12.107143  
16    11.994350  
17    12.268156  
18    11.771429  
19     9.531250  
20    12.000000  
21    11.000000  
22     5.000000  
Name: G3, dtype: float64
```

```
In [21]: import matplotlib.pyplot as plt  
  
plt.figure(figsize=(10, 6))  
plt.plot(age_gp, marker='o', linestyle='--')  
plt.xlabel('Age')  
plt.ylabel('Grades')  
plt.title('Final Grades by Age')  
plt.xticks(age_gp.index)  
plt.grid()  
  
# Save the figure as an image  
plt.savefig("final_grades_by_age.png")  
  
plt.show()
```



```
In [24]: counts = pd.crosstab(df.sex, df.G3)
counts
```

```
Out[24]:
```

	G3	0	1	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
sex																		
F		7	0	0	0	7	20	16	52	54	44	54	41	32	23	21	11	1
M		8	1	1	3	3	15	19	45	50	28	28	22	17	13	8	4	1

```
In [26]: bar_width = 0.35
grades = df.G3.value_counts().sort_index().index

r1 = np.arange(len(grades))
r2 = [x + bar_width for x in r1]

fig, ax = plt.subplots(figsize=(12, 6))

# Female bars
bars1 = ax.bar(r1, counts.loc['F'], color='#ff9999', width=bar_width, edgecolor='gr

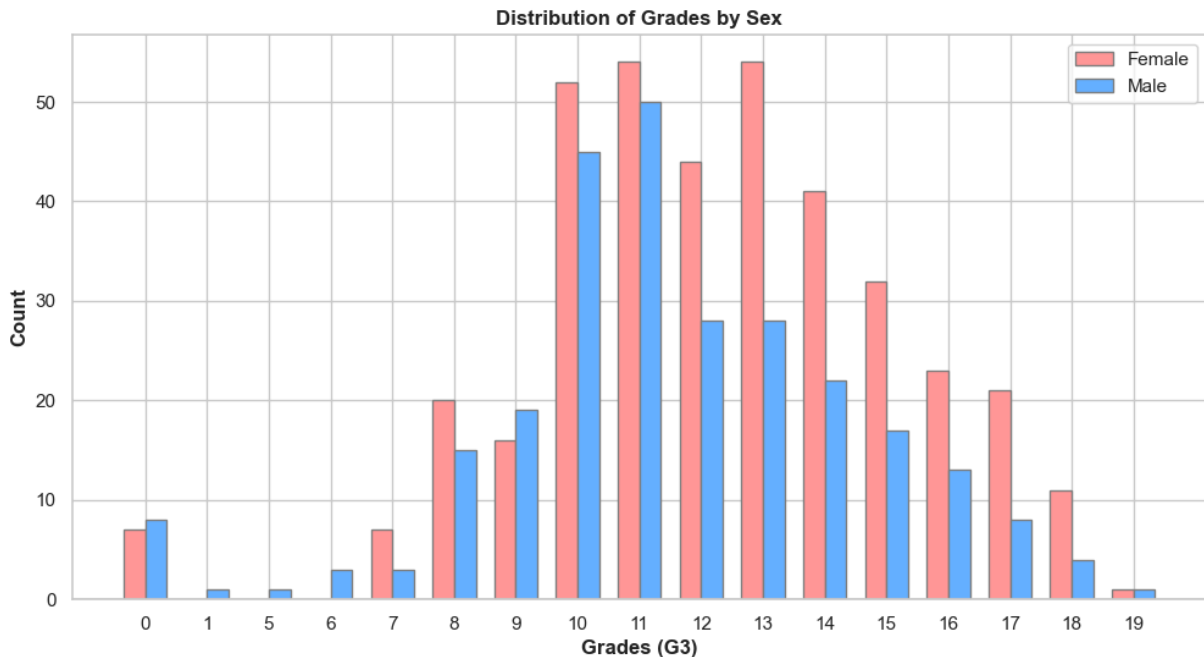
# Male bars
bars2 = ax.bar(r2, counts.loc['M'], color='#66b3ff', width=bar_width, edgecolor='gr

# Labels and title
ax.set_xlabel('Grades (G3)', fontweight='bold')
ax.set_ylabel('Count', fontweight='bold')
ax.set_title('Distribution of Grades by Sex', fontweight='bold')
ax.set_xticks([r + bar_width/2 for r in range(len(grades))])
ax.set_xticklabels(grades)
```

```
ax.legend()

# Save the figure as an image
plt.savefig("grades_by_sex.png") # Can also use .jpg, .pdf, etc.

# Show the plot
plt.show()
```



```
In [27]: mthr_job = df.Mjob.value_counts()
mthr_job
```

```
Out[27]: Mjob
other      258
services   136
at_home    135
teacher     72
health      48
Name: count, dtype: int64
```

```
In [28]: fthr_job = df.Fjob.value_counts()
fthr_job
```

```
Out[28]: Fjob
other      367
services   181
at_home     42
teacher     36
health      23
Name: count, dtype: int64
```

```
In [30]: fig, ax = plt.subplots(1, 2, figsize=(10, 6))

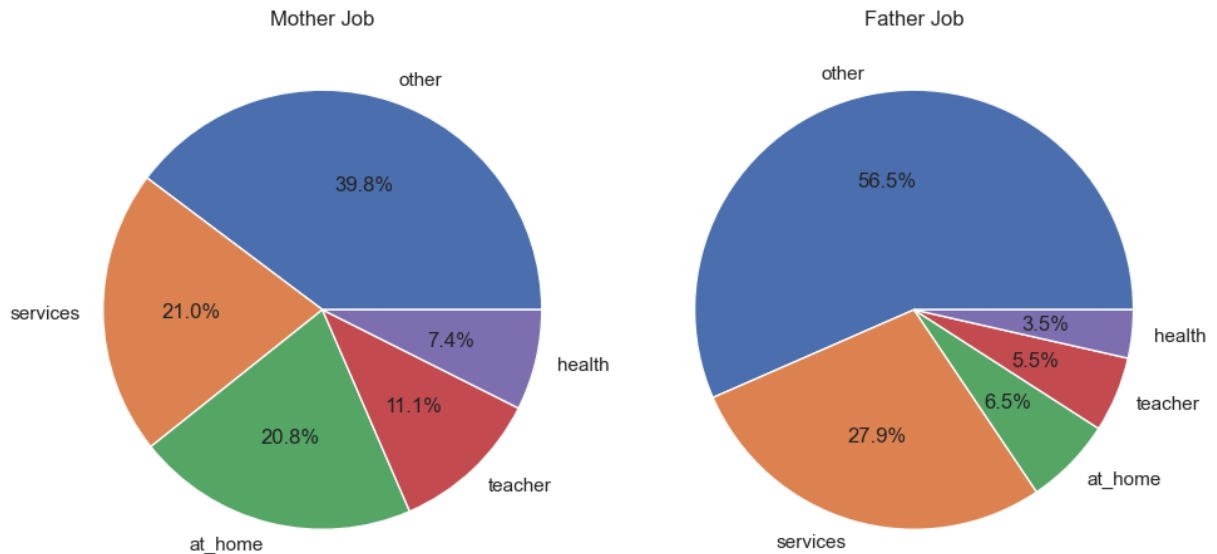
# Mother Job pie chart
ax[0].pie(mthr_job, labels=mthr_job.index, autopct='%1.1f%%')
ax[0].set_title('Mother Job')
```

```
# Father Job pie chart
ax[1].pie(fthr_job, labels=fthr_job.index, autopct='%1.1f%%')
ax[1].set_title('Father Job')

plt.tight_layout() # Adjust layout so titles and labels don't overlap

# Save the figure as an image
plt.savefig("parents_job_piecharts.png") # Can also use .jpg, .pdf, etc.

plt.show()
```



```
In [32]: m_gp = df.groupby('Mjob')['G3'].mean().sort_values()
m_gp
```

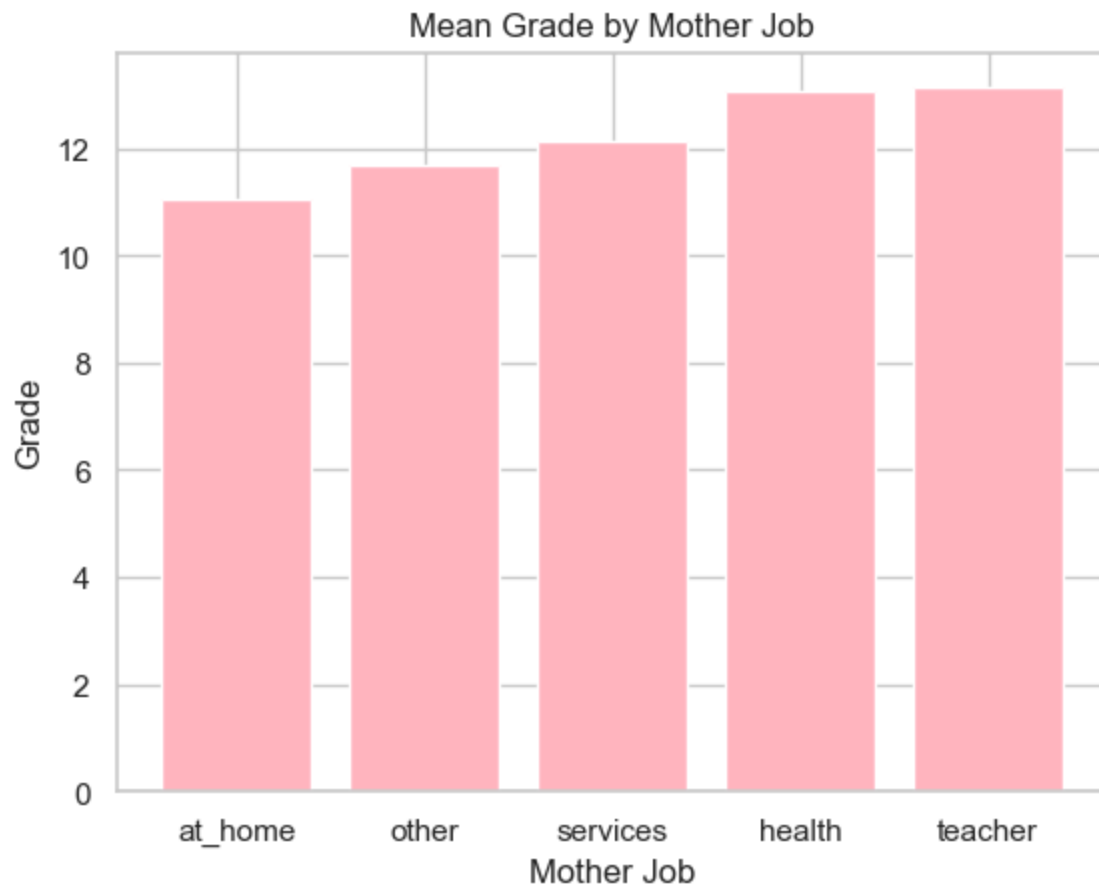
```
Out[32]: Mjob
at_home    11.044444
other      11.670543
services   12.147059
health     13.062500
teacher    13.138889
Name: G3, dtype: float64
```

```
In [33]: f_gp = df.groupby('Fjob')['G3'].mean().sort_values()
f_gp
```

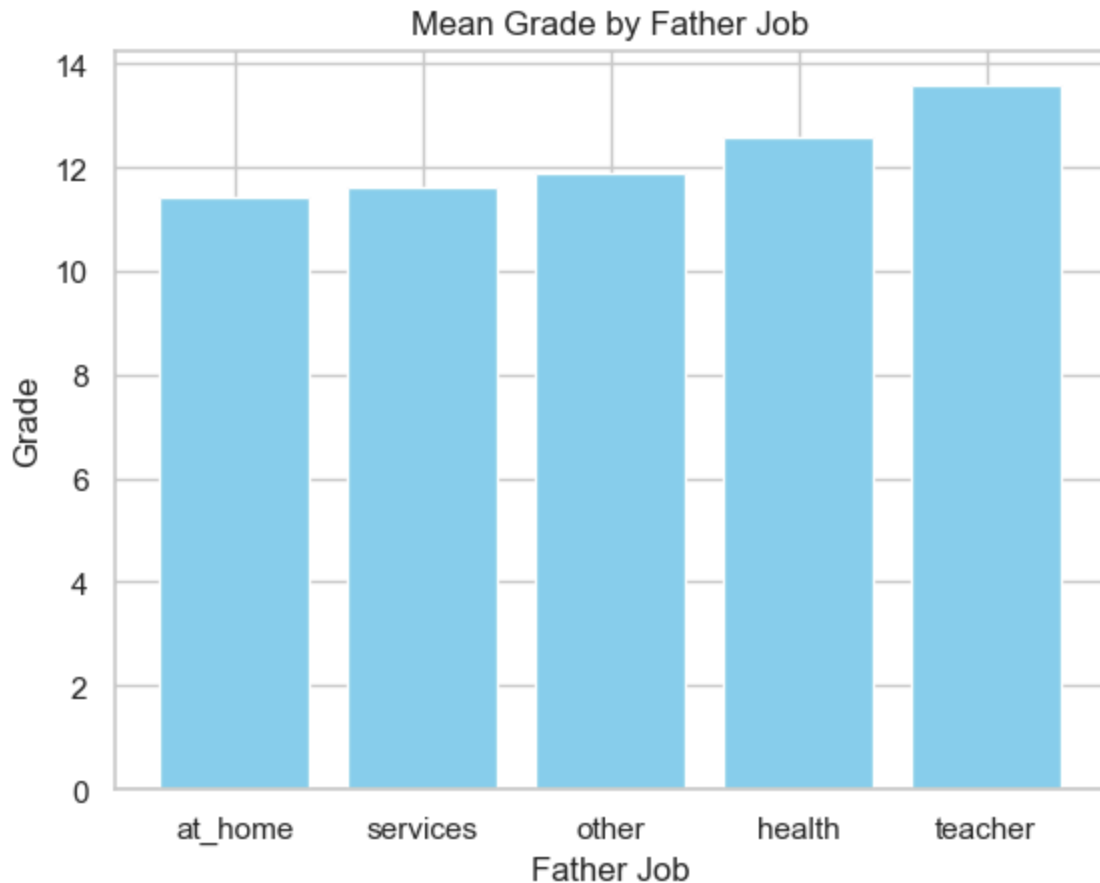
```
Out[33]: Fjob
at_home    11.428571
services   11.629834
other      11.891008
health     12.565217
teacher    13.583333
Name: G3, dtype: float64
```

```
In [34]: plt.bar(m_gp.index, m_gp, color='lightpink')
plt.title("Mean Grade by Mother Job")
plt.xlabel("Mother Job")
plt.ylabel("Grade")
```

```
# Save the figure as an image  
plt.savefig("mean_grade_mother_job.png") # You can also use .jpg, .pdf, etc.  
  
plt.show()
```



```
In [35]: plt.bar(f_gp.index, f_gp, color='skyblue')  
plt.title("Mean Grade by Father Job")  
plt.xlabel("Father Job")  
plt.ylabel("Grade")  
  
# Save the figure as an image  
plt.savefig("mean_grade_father_job.png") # Can also use .jpg, .pdf, etc.  
  
plt.show()
```



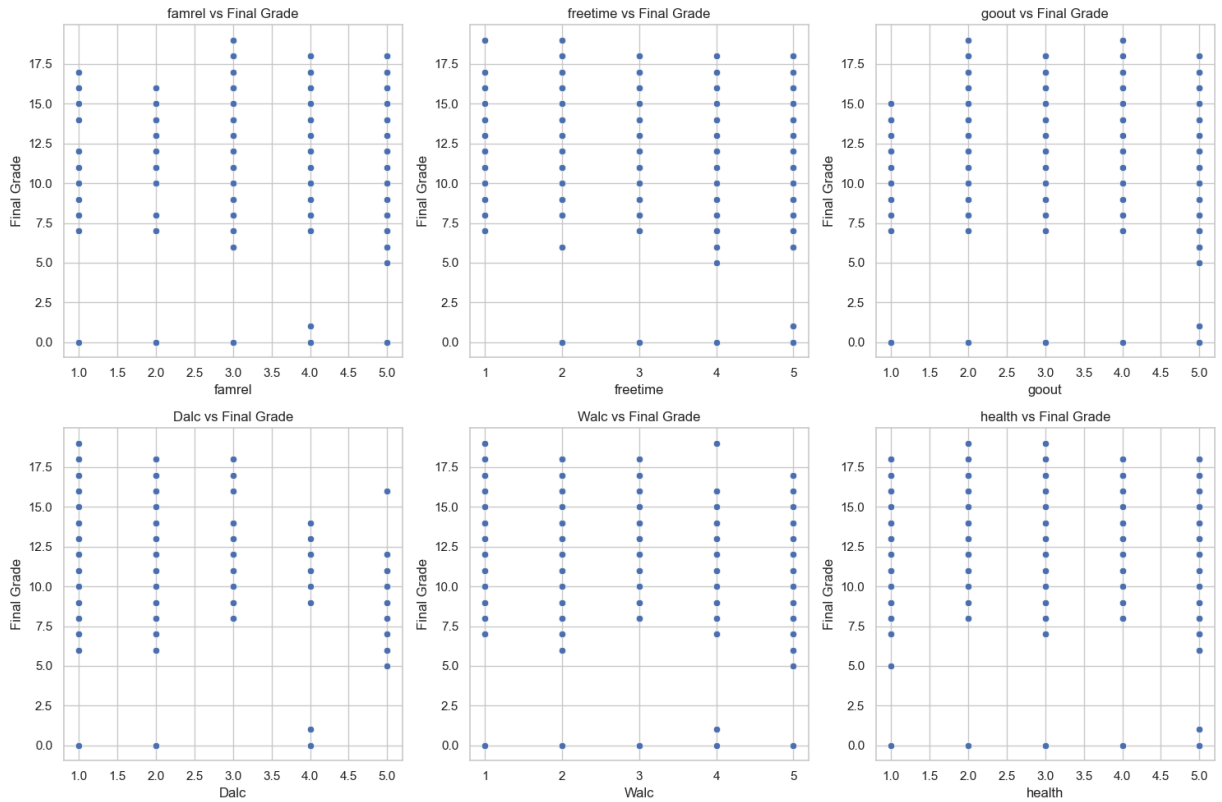
```
In [36]: cols = ["famrel", "freetime", "goout", "Dalc", "Walc", "health"]
fig, ax = plt.subplots(2, 3, figsize=(15, 10))

for i, col in enumerate(cols):
    row = i // 3
    col_idx = i % 3
    sns.scatterplot(x=df[col], y=df.G3, ax=ax[row, col_idx])
    ax[row, col_idx].set_title(f'{col} vs Final Grade')
    ax[row, col_idx].set_xlabel(col)
    ax[row, col_idx].set_ylabel('Final Grade')

plt.tight_layout() # Adjust spacing

# Save the figure as an image
plt.savefig("scatterplots_vs_final_grade.png") # Can also use .jpg, .pdf, etc.

plt.show()
```



```
In [37]: cols = ['schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet']
fig, axes = plt.subplots(4, 2, figsize=(10, 14))

for i, col in enumerate(cols):
    row = i // 2
    col_idx = i % 2
    mean_grades = df.groupby(col)['G3'].mean()
    bar_colors = mean_grades.index.map({'yes': 'lightgreen', 'no': 'lightcoral'})
    mean_grades.plot(kind='bar', ax=axes[row, col_idx], color=bar_colors)
    axes[row, col_idx].set_xlabel(col, fontsize=10, fontweight='bold')
    axes[row, col_idx].set_ylabel('Mean Grade', fontsize=10, fontweight='bold')
    axes[row, col_idx].tick_params(axis='x', rotation=0, labels=12)
    axes[row, col_idx].tick_params(axis='y', labels=12)

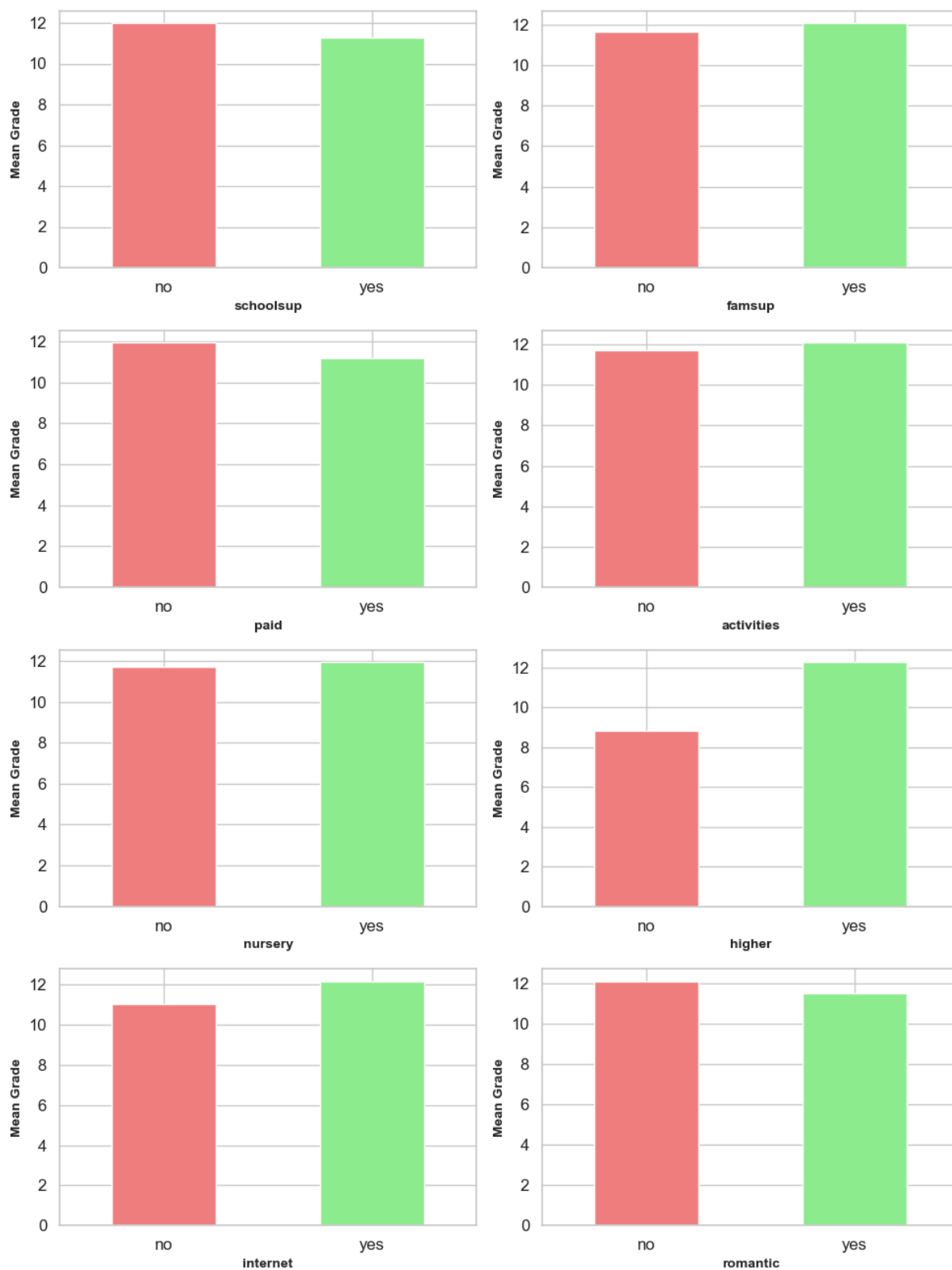
fig.suptitle('Mean Grade Analysis by Binary Categories', fontsize=17, fontweight='b')

plt.tight_layout(rect=[0, 0, 1, 0.97])

# Save the entire figure as an image
plt.savefig("mean_grade_binary_categories.png") # Can also use .jpg, .pdf, etc.

plt.show()
```

Mean Grade Analysis by Binary Categories



```
In [38]: cols = ['studytime', 'traveltime', 'failures', 'absences']
fig, axes = plt.subplots(2, 2, figsize=(10, 14))
for i, col in enumerate(cols):
```

```
row = i // 2
col_idx = i % 2
mean_grades = df.groupby(col)['G3'].mean()

mean_grades.plot(ax=axes[row, col_idx])
axes[row, col_idx].set_xlabel(col, fontsize=10, fontweight='bold')
axes[row, col_idx].set_ylabel('Mean Grade', fontsize=10, fontweight='bold')
axes[row, col_idx].tick_params(axis='x', rotation=0, labels=12)
axes[row, col_idx].tick_params(axis='y', labels=12)

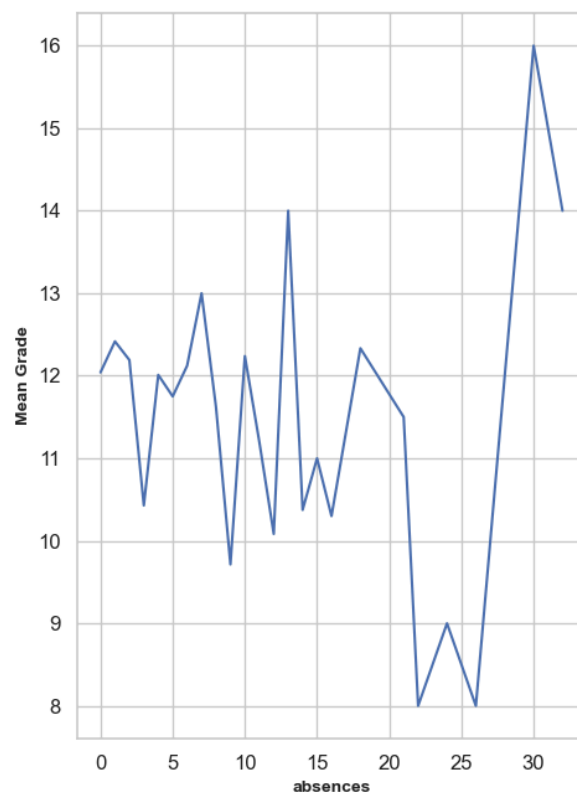
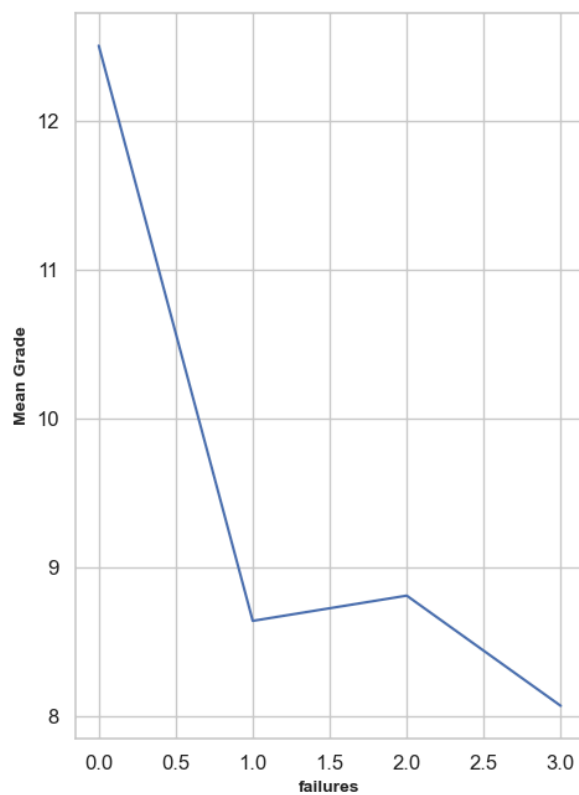
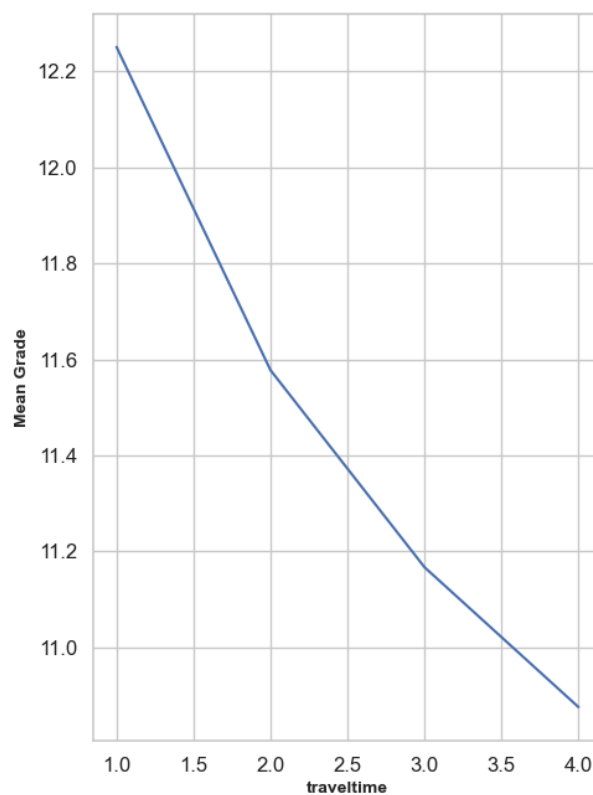
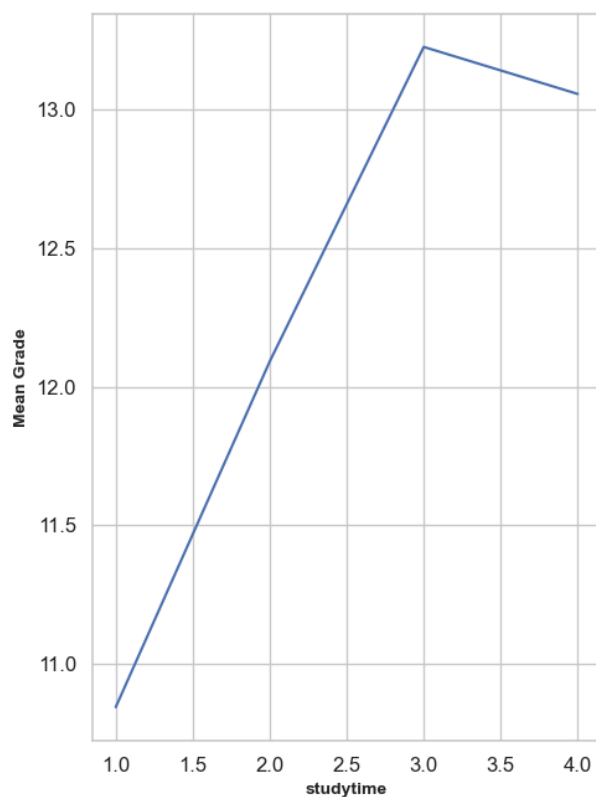
fig.suptitle('Mean Grade Analysis by Different Categories', fontsize=17, fontweight='bold')

plt.tight_layout(rect=[0, 0, 1, 0.97])

# Save the entire figure as an image
plt.savefig("mean_grade_numeric_categories.png") # Can also use .jpg, .pdf, etc.

plt.show()
```

Mean Grade Analysis by Different Categories



```
In [39]: # G3 vs G1
plot1 = sns.lmplot(x='G3', y='G1', data=df, scatter_kws={'alpha':0.3})
plot1.fig.suptitle('G3 vs G1', fontsize=16) # Set title
plt.tight_layout()
plot1.fig.savefig("G3_vs_G1.png") # Save the figure
```

```
plt.close(plot1.fig) # Close the plot to avoid overlap in notebook

# G3 vs G2
plot2 = sns.lmplot(x='G3', y='G2', data=df, scatter_kws={'alpha':0.3})
plot2.fig.suptitle('G3 vs G2', fontsize=16)
plt.tight_layout()
plot2.fig.savefig("G3_vs_G2.png") # Save the figure
plt.close(plot2.fig)
```

```
In [41]: num_cols = []
        for col in df.columns:
            if df[col].dtype != 'object':
                num_cols.append(col)
        num_cols
```

```
Out[41]: ['age',
          'Medu',
          'Fedu',
          'traveltime',
          'studytime',
          'failures',
          'famrel',
          'freetime',
          'goout',
          'Dalc',
          'Walc',
          'health',
          'absences',
          'G1',
          'G2',
          'G3']
```

```
In [42]: corr = df[num_cols].corr()
        corr
```

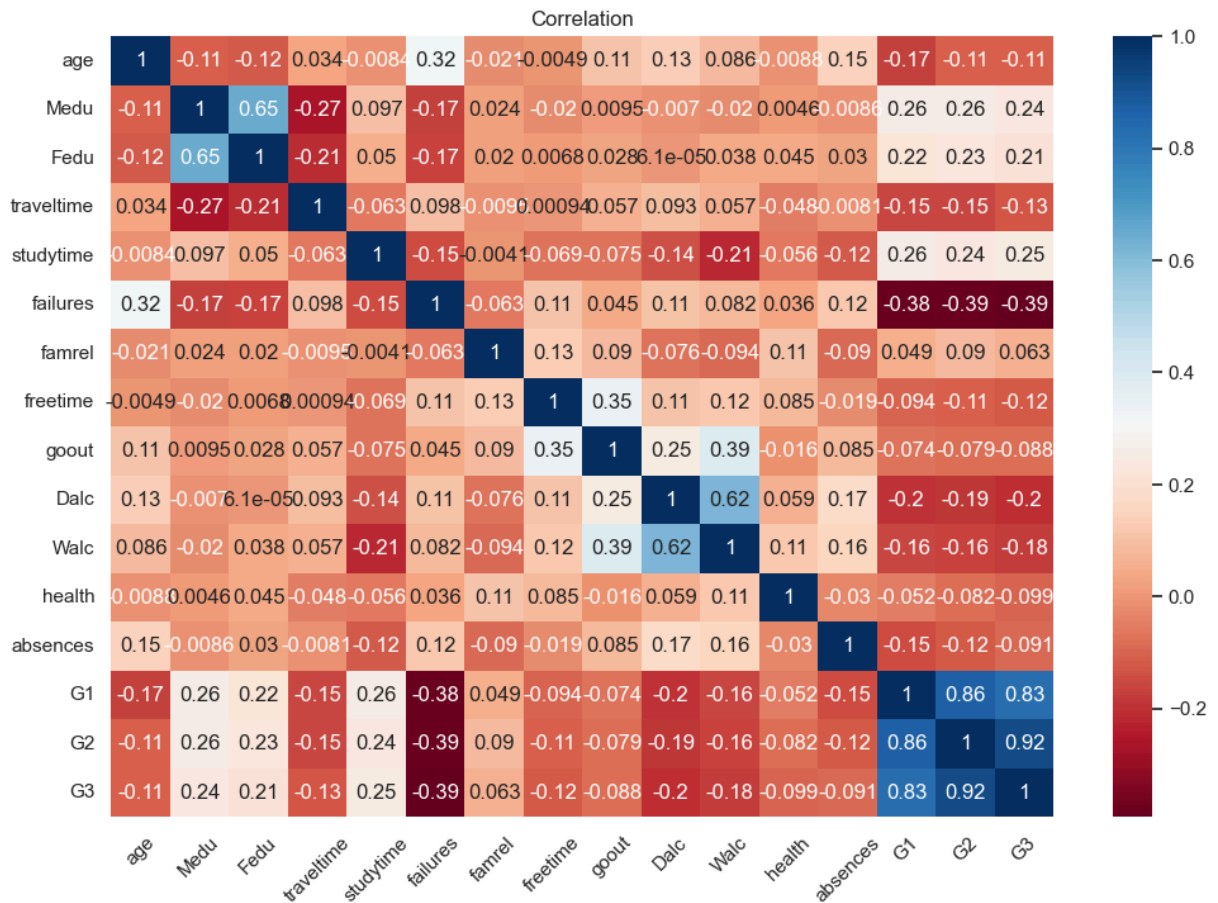
Out[42]:

	age	Medu	Fedu	traveltime	studytime	failures	famrel	fr
age	1.000000	-0.107832	-0.121050	0.034490	-0.008415	0.319968	-0.020559	-0.
Medu	-0.107832	1.000000	0.647477	-0.265079	0.097006	-0.172210	0.024421	-0.
Fedu	-0.121050	0.647477	1.000000	-0.208288	0.050400	-0.165915	0.020256	0.
traveltime	0.034490	-0.265079	-0.208288	1.000000	-0.063154	0.097730	-0.009521	0.
studytime	-0.008415	0.097006	0.050400	-0.063154	1.000000	-0.147441	-0.004127	-0.
failures	0.319968	-0.172210	-0.165915	0.097730	-0.147441	1.000000	-0.062645	0.
famrel	-0.020559	0.024421	0.020256	-0.009521	-0.004127	-0.062645	1.000000	0.
freetime	-0.004910	-0.019686	0.006841	0.000937	-0.068829	0.108995	0.129216	1.
goout	0.112805	0.009536	0.027690	0.057454	-0.075442	0.045078	0.089707	0.
Dalc	0.134768	-0.007018	0.000061	0.092824	-0.137585	0.105949	-0.075767	0.
Walc	0.086357	-0.019766	0.038445	0.057007	-0.214925	0.082266	-0.093511	0.
health	-0.008750	0.004614	0.044910	-0.048261	-0.056433	0.035588	0.109559	0.
absences	0.149998	-0.008577	0.029859	-0.008149	-0.118389	0.122779	-0.089534	-0.
G1	-0.174322	0.260472	0.217501	-0.154120	0.260875	-0.384210	0.048795	-0.
G2	-0.107119	0.264035	0.225139	-0.154489	0.240498	-0.385782	0.089588	-0.
G3	-0.106505	0.240151	0.211800	-0.127173	0.249789	-0.393316	0.063361	-0.



```
In [43]: plt.figure(figsize=(12, 8))
sns.heatmap(corr, annot=True, cmap='RdBu')
plt.title('Correlation')
plt.xticks(rotation=45)

# Save the figure as an image
plt.savefig("correlation_heatmap.png")
plt.show()
```



```
In [44]: import pandas as pd
import statsmodels.api as sm

# 1. Feature Engineering
df['total_alc'] = (df['Dalc'] * 5 + df['Walc'] * 2) / 7
df['has_failed_before'] = (df['failures'] > 0).astype(int)

# 2. Handle Categorical Data
categorical_columns = df.select_dtypes(include=['object']).columns
df_encoded = pd.get_dummies(df, columns=categorical_columns, drop_first=True)

# 3. Define Target (y) and Features (X)
y = df_encoded['G3'].astype(float)
# Drop the target and the redundant columns
X = df_encoded.drop(['G3', 'Dalc', 'Walc', 'failures'], axis=1)

# 4. Final Preparation
X = sm.add_constant(X) # Add intercept
X = X.astype(float)    # Convert all to float (including the new dummies)

# 5. Fit the Initial OLS Model
full_model = sm.OLS(y, X).fit()

print("--- INITIAL FULL MODEL SUMMARY ---")
print(full_model.summary())
```

--- INITIAL FULL MODEL SUMMARY ---

OLS Regression Results

```

=====
Dep. Variable:          G3      R-squared:          0.861
Model:                  OLS      Adj. R-squared:       0.852
Method:                 Least Squares      F-statistic:       94.12
Date:                   Wed, 22 Apr 2026      Prob (F-statistic):   4.38e-232
Time:                   10:58:22      Log-Likelihood:      -1041.2
No. Observations:       649      AIC:                2164.
Df Residuals:           608      BIC:                2348.
Df Model:               40
Covariance Type:        nonrobust
=====

```

```

=====
=
              coef      std err          t      P>|t|      [0.025      0.97
5]
-----
-
const          0.6135      0.958      0.640      0.522      -1.268      2.49
5
age            0.0356      0.048      0.742      0.459      -0.059      0.13
0
Medu          -0.1049      0.070     -1.489      0.137      -0.243      0.03
3
Fedu           0.0594      0.064      0.929      0.353      -0.066      0.18
5
traveltime     0.1384      0.074      1.865      0.063      -0.007      0.28
4
studytime      0.0493      0.066      0.752      0.452      -0.079      0.17
8
famrel        -0.0170      0.054     -0.314      0.753      -0.124      0.08
9
freetime      -0.0496      0.052     -0.948      0.344      -0.152      0.05
3
goout         -0.0224      0.049     -0.461      0.645      -0.118      0.07
3
health        -0.0548      0.036     -1.521      0.129      -0.125      0.01
6
absences       0.0157      0.012      1.336      0.182      -0.007      0.03
9
G1             0.1301      0.037      3.476      0.001      0.057      0.20
4
G2             0.8630      0.035     24.674      0.000      0.794      0.93
2
total_alc     -0.0680      0.064     -1.070      0.285      -0.193      0.05
7
has_failed_before -0.5445      0.165     -3.292      0.001      -0.869     -0.22
0
school_MS     -0.1781      0.127     -1.398      0.163      -0.428      0.07
2
sex_M         -0.1286      0.117     -1.102      0.271      -0.358      0.10
1
address_U      0.1106      0.122      0.905      0.366      -0.129      0.35
1
famsize_LE3    0.0109      0.115      0.095      0.924      -0.214      0.23
6

```

Pstatus_T 1	-0.0868	0.162	-0.536	0.592	-0.405	0.23
Mjob_health 4	0.2913	0.251	1.161	0.246	-0.201	0.78
Mjob_other 1	-0.0870	0.141	-0.615	0.539	-0.365	0.19
Mjob_services 7	0.1747	0.174	1.002	0.317	-0.168	0.51
Mjob_teacher 4	0.2231	0.234	0.952	0.342	-0.237	0.68
Fjob_health 3	-0.4462	0.351	-1.272	0.204	-1.135	0.24
Fjob_other 2	-0.3366	0.213	-1.581	0.114	-0.755	0.08
Fjob_services 6	-0.4651	0.224	-2.079	0.038	-0.905	-0.02
Fjob_teacher 8	-0.5384	0.314	-1.714	0.087	-1.155	0.07
reason_home 6	-0.0854	0.133	-0.642	0.521	-0.346	0.17
reason_other 4	-0.3707	0.171	-2.162	0.031	-0.707	-0.03
reason_reputation 3	-0.1805	0.139	-1.298	0.195	-0.454	0.09
guardian_mother 6	-0.0276	0.124	-0.223	0.824	-0.271	0.21
guardian_other 6	0.2495	0.248	1.006	0.315	-0.237	0.73
schoolsup_yes 4	-0.1836	0.172	-1.067	0.286	-0.522	0.15
famsup_yes 1	0.0922	0.106	0.867	0.386	-0.117	0.30
paid_yes 8	-0.1754	0.216	-0.813	0.417	-0.599	0.24
activities_yes 5	2.799e-05	0.104	0.000	1.000	-0.205	0.20
nursery_yes 7	-0.0818	0.127	-0.646	0.518	-0.330	0.16
higher_yes 9	0.2029	0.181	1.118	0.264	-0.153	0.55
internet_yes 8	0.1052	0.129	0.817	0.414	-0.148	0.35
romantic_yes 6	-0.0443	0.107	-0.413	0.680	-0.255	0.16

```

=====
Omnibus:                418.942    Durbin-Watson:                1.870
Prob(Omnibus):          0.000    Jarque-Bera (JB):            7939.419
Skew:                   -2.545    Prob(JB):                     0.00
Kurtosis:               19.362    Cond. No.                     501.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [45]: # Convert boolean dummies to float for the model
X = X.astype(float)

# Fit the Initial OLS Model
full_model = sm.OLS(y, X).fit()

# View the full results
print("--- INITIAL FULL MODEL SUMMARY ---")
print(full_model.summary())
```

--- INITIAL FULL MODEL SUMMARY ---

OLS Regression Results

```

=====
Dep. Variable:          G3      R-squared:          0.861
Model:                  OLS      Adj. R-squared:       0.852
Method:                 Least Squares      F-statistic:       94.12
Date:                   Wed, 22 Apr 2026      Prob (F-statistic): 4.38e-232
Time:                   10:58:34      Log-Likelihood:    -1041.2
No. Observations:       649      AIC:              2164.
Df Residuals:           608      BIC:              2348.
Df Model:                40
Covariance Type:        nonrobust
=====

```

```

=====
=
              coef      std err          t      P>|t|      [0.025      0.97
5]
-----
-
const          0.6135      0.958      0.640      0.522      -1.268      2.49
5
age            0.0356      0.048      0.742      0.459      -0.059      0.13
0
Medu          -0.1049      0.070     -1.489      0.137      -0.243      0.03
3
Fedu           0.0594      0.064      0.929      0.353      -0.066      0.18
5
traveltime     0.1384      0.074      1.865      0.063      -0.007      0.28
4
studytime      0.0493      0.066      0.752      0.452      -0.079      0.17
8
famrel        -0.0170      0.054     -0.314      0.753      -0.124      0.08
9
freetime      -0.0496      0.052     -0.948      0.344      -0.152      0.05
3
goout         -0.0224      0.049     -0.461      0.645      -0.118      0.07
3
health        -0.0548      0.036     -1.521      0.129      -0.125      0.01
6
absences       0.0157      0.012      1.336      0.182      -0.007      0.03
9
G1             0.1301      0.037      3.476      0.001      0.057      0.20
4
G2             0.8630      0.035     24.674      0.000      0.794      0.93
2
total_alc     -0.0680      0.064     -1.070      0.285      -0.193      0.05
7
has_failed_before -0.5445      0.165     -3.292      0.001      -0.869     -0.22
0
school_MS     -0.1781      0.127     -1.398      0.163      -0.428      0.07
2
sex_M         -0.1286      0.117     -1.102      0.271      -0.358      0.10
1
address_U      0.1106      0.122      0.905      0.366      -0.129      0.35
1
famsize_LE3    0.0109      0.115      0.095      0.924      -0.214      0.23
6

```

Pstatus_T 1	-0.0868	0.162	-0.536	0.592	-0.405	0.23
Mjob_health 4	0.2913	0.251	1.161	0.246	-0.201	0.78
Mjob_other 1	-0.0870	0.141	-0.615	0.539	-0.365	0.19
Mjob_services 7	0.1747	0.174	1.002	0.317	-0.168	0.51
Mjob_teacher 4	0.2231	0.234	0.952	0.342	-0.237	0.68
Fjob_health 3	-0.4462	0.351	-1.272	0.204	-1.135	0.24
Fjob_other 2	-0.3366	0.213	-1.581	0.114	-0.755	0.08
Fjob_services 6	-0.4651	0.224	-2.079	0.038	-0.905	-0.02
Fjob_teacher 8	-0.5384	0.314	-1.714	0.087	-1.155	0.07
reason_home 6	-0.0854	0.133	-0.642	0.521	-0.346	0.17
reason_other 4	-0.3707	0.171	-2.162	0.031	-0.707	-0.03
reason_reputation 3	-0.1805	0.139	-1.298	0.195	-0.454	0.09
guardian_mother 6	-0.0276	0.124	-0.223	0.824	-0.271	0.21
guardian_other 6	0.2495	0.248	1.006	0.315	-0.237	0.73
schoolsup_yes 4	-0.1836	0.172	-1.067	0.286	-0.522	0.15
famsup_yes 1	0.0922	0.106	0.867	0.386	-0.117	0.30
paid_yes 8	-0.1754	0.216	-0.813	0.417	-0.599	0.24
activities_yes 5	2.799e-05	0.104	0.000	1.000	-0.205	0.20
nursery_yes 7	-0.0818	0.127	-0.646	0.518	-0.330	0.16
higher_yes 9	0.2029	0.181	1.118	0.264	-0.153	0.55
internet_yes 8	0.1052	0.129	0.817	0.414	-0.148	0.35
romantic_yes 6	-0.0443	0.107	-0.413	0.680	-0.255	0.16

```

=====
Omnibus:                418.942    Durbin-Watson:                1.870
Prob(Omnibus):          0.000    Jarque-Bera (JB):            7939.419
Skew:                   -2.545    Prob(JB):                     0.00
Kurtosis:               19.362    Cond. No.                     501.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [46]: def automated_backward_elimination(X, y, threshold=0.05):
    features = X.columns.tolist()
    iteration = 1

    while len(features) > 0:
        # Fit the model with current features
        model = sm.OLS(y, X[features].astype(float)).fit()
        # Get p-values for all features (excluding the constant for the check)
        p_values = model.pvalues

        # Identify the feature with the maximum p-value
        max_p = p_values.max()
        feature_to_remove = p_values.idxmax()

        if max_p > threshold:
            print(f"Iteration {iteration}: Removing '{feature_to_remove}' (p-value:
            features.remove(feature_to_remove)
            iteration += 1
        else:
            # All remaining variables are significant
            break

    return model

# Running the loop on your encoded dataframe
# Note: 'X' should be your dataframe with the constant added and G1/G2 included
final_significant_model = automated_backward_elimination(X, y)

# Print the final result
print("\n" + "="*30)
print("FINAL SIGNIFICANT MODEL SUMMARY")
print("="*30)
print(final_significant_model.summary())
```

Iteration 1: Removing 'activities_yes' (p-value: 0.9998)
 Iteration 2: Removing 'famsize_LE3' (p-value: 0.9242)
 Iteration 3: Removing 'guardian_mother' (p-value: 0.8224)
 Iteration 4: Removing 'famrel' (p-value: 0.7588)
 Iteration 5: Removing 'romantic_yes' (p-value: 0.6725)
 Iteration 6: Removing 'goout' (p-value: 0.6206)
 Iteration 7: Removing 'Pstatus_T' (p-value: 0.5744)
 Iteration 8: Removing 'const' (p-value: 0.5634)
 Iteration 9: Removing 'Mjob_other' (p-value: 0.5635)
 Iteration 10: Removing 'nursery_yes' (p-value: 0.5529)
 Iteration 11: Removing 'reason_home' (p-value: 0.5156)
 Iteration 12: Removing 'internet_yes' (p-value: 0.5056)
 Iteration 13: Removing 'studytime' (p-value: 0.4507)
 Iteration 14: Removing 'paid_yes' (p-value: 0.3955)
 Iteration 15: Removing 'famsup_yes' (p-value: 0.3281)
 Iteration 16: Removing 'schoolsup_yes' (p-value: 0.3354)
 Iteration 17: Removing 'reason_reputation' (p-value: 0.3058)
 Iteration 18: Removing 'Fedu' (p-value: 0.2789)
 Iteration 19: Removing 'freetime' (p-value: 0.2962)
 Iteration 20: Removing 'address_U' (p-value: 0.2426)
 Iteration 21: Removing 'Fjob_health' (p-value: 0.2429)
 Iteration 22: Removing 'higher_yes' (p-value: 0.2017)
 Iteration 23: Removing 'Fjob_other' (p-value: 0.2059)
 Iteration 24: Removing 'Fjob_teacher' (p-value: 0.4385)
 Iteration 25: Removing 'Fjob_services' (p-value: 0.3889)
 Iteration 26: Removing 'Medu' (p-value: 0.1706)
 Iteration 27: Removing 'Mjob_teacher' (p-value: 0.3338)
 Iteration 28: Removing 'Mjob_health' (p-value: 0.2744)
 Iteration 29: Removing 'Mjob_services' (p-value: 0.2908)
 Iteration 30: Removing 'guardian_other' (p-value: 0.2017)
 Iteration 31: Removing 'health' (p-value: 0.1801)
 Iteration 32: Removing 'total_alc' (p-value: 0.1387)
 Iteration 33: Removing 'age' (p-value: 0.2821)
 Iteration 34: Removing 'school_MS' (p-value: 0.1557)
 Iteration 35: Removing 'traveltime' (p-value: 0.0549)
 Iteration 36: Removing 'sex_M' (p-value: 0.0666)

=====
 FINAL SIGNIFICANT MODEL SUMMARY
 =====

OLS Regression Results

```
=====
===
Dep. Variable:          G3      R-squared (uncentered):          0.
990
Model:                  OLS      Adj. R-squared (uncentered):          0.
990
Method:                 Least Squares      F-statistic:          1.262e
+04
Date:                   Wed, 22 Apr 2026      Prob (F-statistic):
0.00
Time:                   10:58:46      Log-Likelihood:          -106
0.5
No. Observations:       649      AIC:          21
31.
Df Residuals:           644      BIC:          21
```

```

53.
Df Model:                    5
Covariance Type:            nonrobust
=====
=
              coef      std err          t      P>|t|      [0.025      0.97
5]
-----
-
absences          0.0231      0.010       2.206      0.028      0.003      0.04
4
G1                0.1499      0.034       4.455      0.000      0.084      0.21
6
G2                0.8832      0.033      26.727      0.000      0.818      0.94
8
has_failed_before -0.4282      0.135      -3.182      0.002     -0.692     -0.16
4
reason_other     -0.4202      0.154      -2.722      0.007     -0.723     -0.11
7
=====
Omnibus:            443.587   Durbin-Watson:           1.869
Prob(Omnibus):      0.000   Jarque-Bera (JB):       9325.055
Skew:               -2.725   Prob(JB):               0.00
Kurtosis:           20.752   Cond. No.               54.3
=====

```

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

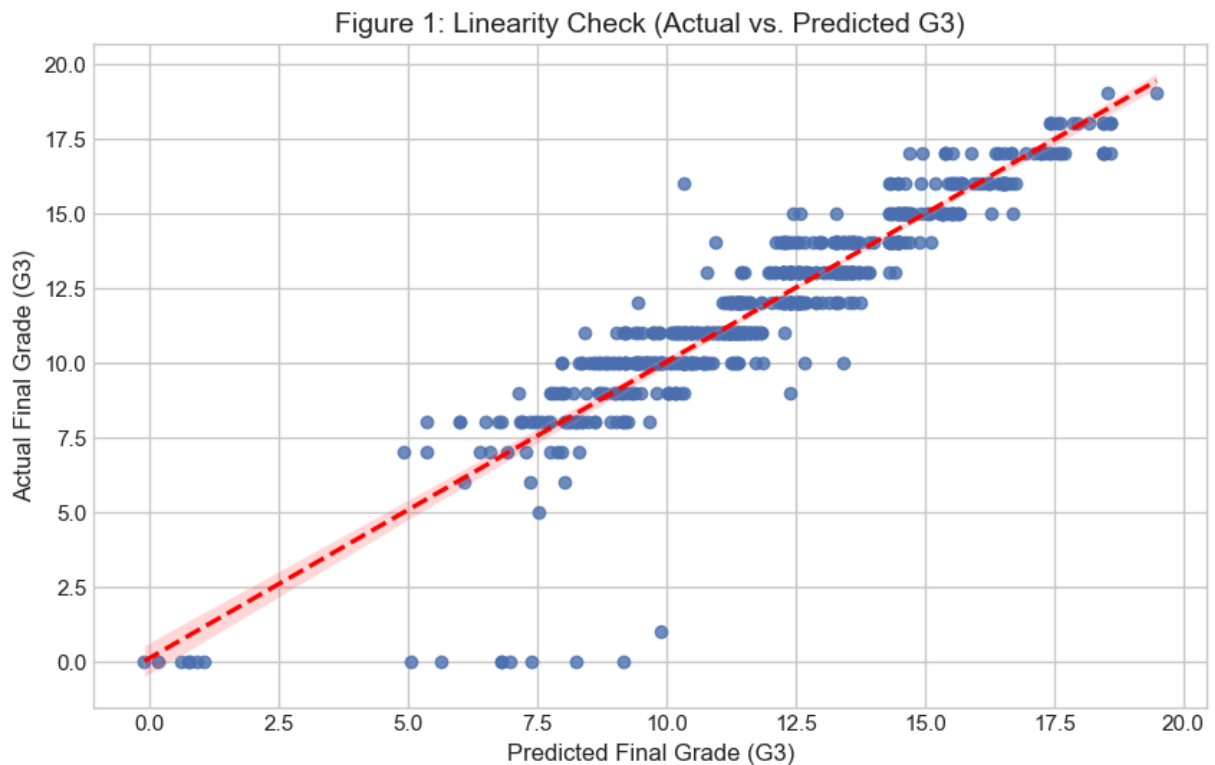
In [47]: # 1. MANUALLY DEFINE THE FINAL FEATURES (From your successful loop)
# This fixes the NameError
final_features = ['absences', 'G1', 'G2', 'has_failed_before', 'reason_other']

# 2. PREPARE PREDICTIONS
# X is your encoded dataframe from the previous steps
y_pred = final_significant_model.predict(X[final_features])
residuals = final_significant_model.resid

# Set professional style
plt.style.use('seaborn-v0_8-whitegrid')

# 3. LINEARITY CHECK
plt.figure(figsize=(10, 6))
sns.regplot(x=y_pred, y=y, lowess=False, line_kws={'color': 'red', 'ls': '--'})
plt.title('Figure 1: Linearity Check (Actual vs. Predicted G3)', fontsize=14)
plt.xlabel('Predicted Final Grade (G3)', fontsize=12)
plt.ylabel('Actual Final Grade (G3)', fontsize=12)
plt.savefig('diagnostic_linearity.png', dpi=300, bbox_inches='tight')
plt.show()

```



```
In [73]: import matplotlib.pyplot as plt
import seaborn as sns

# Using the residuals from your final model
residuals = final_significant_model.resid

# Set the visual style
sns.set(style="whitegrid")
plt.figure(figsize=(10, 6))

# Create the histogram with a KDE curve
sns.histplot(residuals, kde=True, color='teal', bins=30)

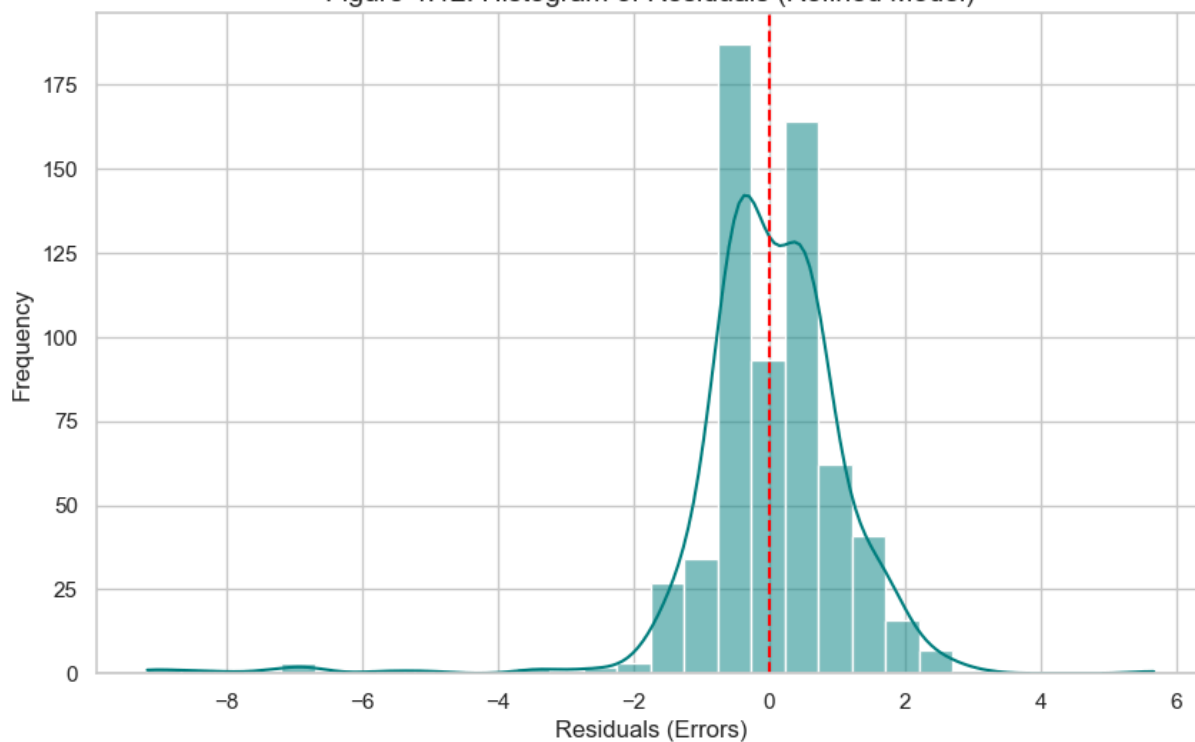
# Add Labels and title
plt.title('Figure 4.12: Histogram of Residuals (Refined Model)', fontsize=14)
plt.xlabel('Residuals (Errors)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)

# Add a vertical line at zero to show the center
plt.axvline(x=0, color='red', linestyle='--')

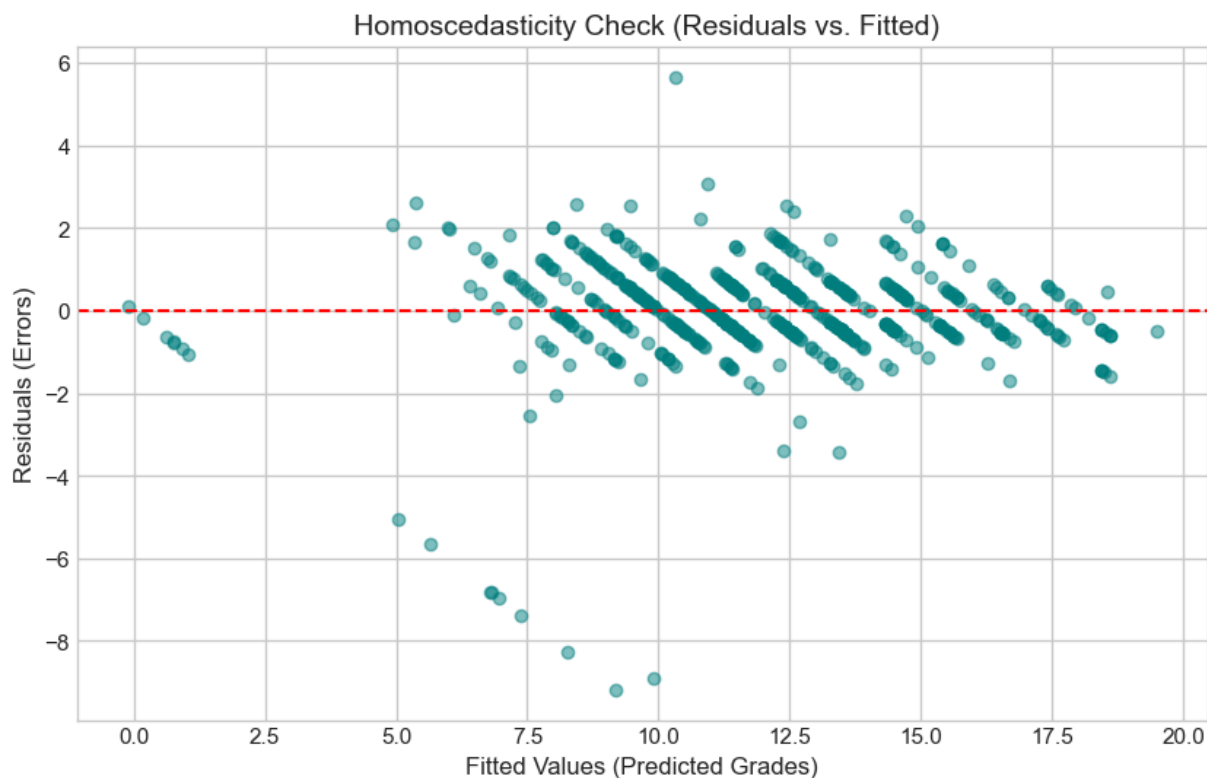
# Save the figure to your current folder
# You can change the extension to .png, .jpg, or .pdf as needed
plt.savefig('histogram_of_residuals.png', dpi=300, bbox_inches='tight')

plt.show()
```

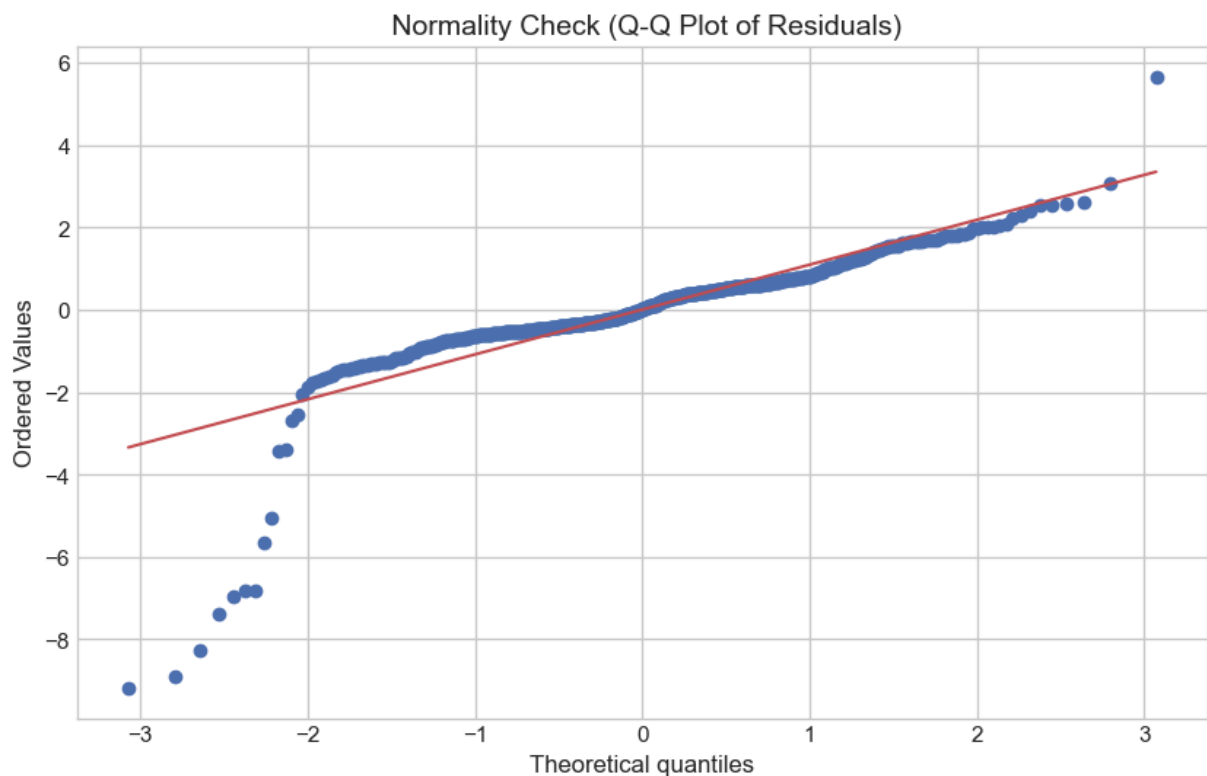
Figure 4.12: Histogram of Residuals (Refined Model)



```
In [49]: # 4. HOMOSCEDASTICITY CHECK
plt.figure(figsize=(10, 6))
plt.scatter(y_pred, residuals, alpha=0.5, color='teal')
plt.axhline(0, color='red', linestyle='--')
plt.title('Homoscedasticity Check (Residuals vs. Fitted)', fontsize=14)
plt.xlabel('Fitted Values (Predicted Grades)', fontsize=12)
plt.ylabel('Residuals (Errors)', fontsize=12)
plt.savefig('diagnostic_homoscedasticity.png', dpi=300, bbox_inches='tight')
plt.show()
```



```
In [50]: # 5. NORMALITY CHECK (Q-Q Plot)
plt.figure(figsize=(10, 6))
stats.probplot(residuals, dist="norm", plot=plt)
plt.title(' Normality Check (Q-Q Plot of Residuals)', fontsize=14)
plt.savefig('diagnostic_normality.png', dpi=300, bbox_inches='tight')
plt.show()
```



```
In [52]: # 6. MULTICOLLINEARITY (VIF)
vif_data = pd.DataFrame()
vif_data["Feature"] = final_features
vif_data["VIF"] = [variance_inflation_factor(X[final_features].values, i)
                    for i in range(len(final_features))]

print("\n--- Variance Inflation Factor (VIF) ---")
print(vif_data.round(2))
```

```
--- Variance Inflation Factor (VIF) ---
      Feature      VIF
0      absences    1.60
1              G1   65.17
2              G2   65.10
3  has_failed_before  1.17
4      reason_other  1.11
```

```
In [53]: import pandas as pd
import numpy as np
import statsmodels.api as sm
import scipy.stats as stats
from statsmodels.stats.diagnostic import het_breuschpagan

# 1. ENSURE DATA IS DEFINED (Using your final selection)
final_features = ['absences', 'G1', 'G2', 'has_failed_before', 'reason_other']

# We need to make sure X and y exist in this cell
y = df_encoded['G3'].astype(float)
X_final = df_encoded[final_features].astype(float)

# 2. RE-GENERATE RESIDUALS
# (In case they were lost when you re-ran the full model)
# We add the constant here because the BP test requires it
X_diag = sm.add_constant(X_final)
temp_model = sm.OLS(y, X_diag).fit()
residuals = temp_model.resid

# 3. RUN THE DIAGNOSTIC TESTS
# Breusch-Pagan for Homoscedasticity
_, bp_pvalue, _, _ = het_breuschpagan(residuals, X_diag)

# Shapiro-Wilk for Normality
_, shapiro_pvalue = stats.shapiro(residuals)

print("--- Final Diagnostic Test Results ---")
print(f"Breusch-Pagan p-value: {bp_pvalue:.4f}")
print(f"Shapiro-Wilk p-value: {shapiro_pvalue:.4f}")

if bp_pvalue < 0.05:
    print("\nNote: Heteroscedasticity detected. Consider using Robust Standard Errors")
if shapiro_pvalue < 0.05:
    print("Note: Residuals are not normal. Justify using the Central Limit Theorem")
```

--- Final Diagnostic Test Results ---

Breusch-Pagan p-value: 0.0000

Shapiro-Wilk p-value: 0.0000

Note: Heteroscedasticity detected. Consider using Robust Standard Errors in your the sis.

Note: Residuals are not normal. Justify using the Central Limit Theorem (N=649).

```
In [54]: from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
import numpy as np

# 1. Define your final features for a fair comparison
# We use the same features for both models
X_final = X[final_features]
y_target = y

# 2. Split the data
X_train, X_test, y_train, y_test = train_test_split(X_final, y_target, test_size=0.

# 3. Fit OLS Model (on training data)
ols_model = sm.OLS(y_train, X_train).fit()
ols_pred = ols_model.predict(X_test)

# 4. Fit Random Forest Model (on training data)
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)

# 5. Calculate Metrics
def get_metrics(y_true, y_pred, model_name):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    return {f"{model_name} MSE": round(mse, 4), f"{model_name} RMSE": round(rmse, 4

ols_results = get_metrics(y_test, ols_pred, "OLS")
rf_results = get_metrics(y_test, rf_pred, "Random Forest")

# Display Results
print("--- MODEL COMPARISON ---")
print(ols_results)
print(rf_results)

--- MODEL COMPARISON ---
{'OLS MSE': 1.3002, 'OLS RMSE': np.float64(1.1403)}
{'Random Forest MSE': 1.5876, 'Random Forest RMSE': np.float64(1.26)}
```

```
In [55]: from sklearn.metrics import r2_score

# Calculate R-squared for both models on the test set
ols_r2 = r2_score(y_test, ols_pred)
rf_r2 = r2_score(y_test, rf_pred)

print(f"OLS Test R-squared: {ols_r2:.4f}")
print(f"Random Forest Test R-squared: {rf_r2:.4f}")
```

OLS Test R-squared: 0.8667

Random Forest Test R-squared: 0.8372

```
In [56]: from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import mean_squared_error, r2_score

print("Step 1: Preparing data...")
# Use your finalized 5 features
final_features = ['absences', 'G1', 'G2', 'has_failed_before', 'reason_other']
X_final = df_encoded[final_features].astype(float)
y_target = df_encoded['G3'].astype(float)

X_train, X_test, y_train, y_test = train_test_split(X_final, y_target, test_size=0.

print("Step 2: Starting Hyperparameter Tuning (This may take 10-20 seconds)...")
param_grid = {
    'max_depth': [3, 5, 10],
    'min_samples_leaf': [2, 5, 10],
    'n_estimators': [100]
}

grid_search = GridSearchCV(
    RandomForestRegressor(random_state=42),
    param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("Step 3: Calculating final metrics...")
best_rf_model = grid_search.best_estimator_
best_rf_pred = best_rf_model.predict(X_test)

# Metrics
pruned_rf_rmse = np.sqrt(mean_squared_error(y_test, best_rf_pred))
pruned_rf_r2 = r2_score(y_test, best_rf_pred)

print("\n" + "="*30)
print("  PRUNED RF FINAL RESULTS  ")
print("="*30)
print(f"Best Max Depth: {grid_search.best_params_['max_depth']}")
print(f"Best Min Samples Leaf: {grid_search.best_params_['min_samples_leaf']}")
print(f"RMSE: {pruned_rf_rmse:.4f}")
print(f"R-squared: {pruned_rf_r2:.4f}")
print("="*30)
```

Step 1: Preparing data...

Step 2: Starting Hyperparameter Tuning (This may take 10-20 seconds)...

Step 3: Calculating final metrics...

```
=====
PRUNED RF FINAL RESULTS
=====
Best Max Depth: 5
Best Min Samples Leaf: 5
RMSE: 1.2059
R-squared: 0.8509
=====
```

```
In [58]: # 5. Calculate Metrics (Updated)
from sklearn.metrics import mean_squared_error, mean_absolute_error
def get_metrics(y_true, y_pred, model_name):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred) # Added MAE

    return {
        f"{model_name} MSE": round(mse, 4),
        f"{model_name} RMSE": round(rmse, 4),
        f"{model_name} MAE": round(mae, 4) # Included in return
    }

ols_results = get_metrics(y_test, ols_pred, "OLS")
rf_results = get_metrics(y_test, rf_pred, "Random Forest")

# Display Results
print("--- MODEL COMPARISON ---")
print(ols_results)
print(rf_results)

--- MODEL COMPARISON ---
{'OLS MSE': 1.3002, 'OLS RMSE': np.float64(1.1403), 'OLS MAE': 0.718}
{'Random Forest MSE': 1.5876, 'Random Forest RMSE': np.float64(1.26), 'Random Forest MAE': 0.7551}
```

```
In [61]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# 1. Get feature importances from your trained rf_model
importances = rf_model.feature_importances_
feature_names = X_final.columns

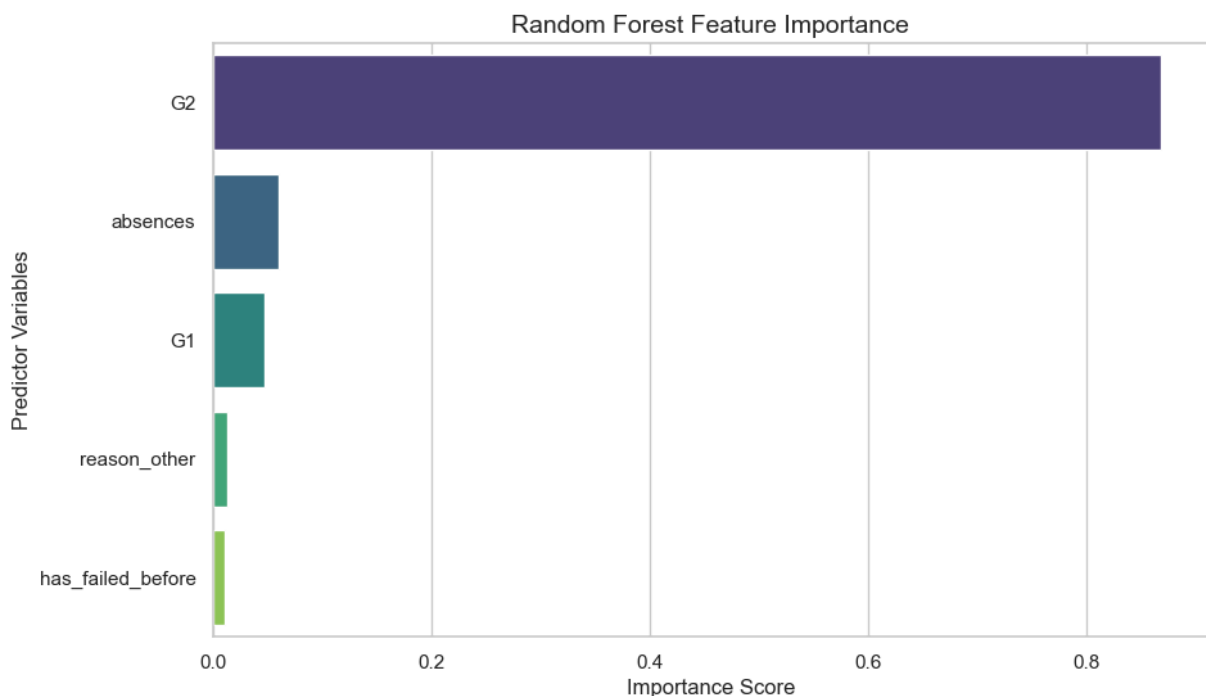
# 2. Create a DataFrame for easy plotting
feature_importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

# Updated plotting code to avoid the FutureWarning
plt.figure(figsize=(10, 6))
sns.set(style="whitegrid")
```

```
# Assigning Feature to hue and setting Legend to False as requested by the warning
sns.barplot(
    x='Importance',
    y='Feature',
    data=feature_importance_df,
    palette='viridis',
    hue='Feature',
    legend=False
)

plt.title('Random Forest Feature Importance', fontsize=14)
plt.xlabel('Importance Score', fontsize=12)
plt.ylabel('Predictor Variables', fontsize=12)

plt.savefig('feature_importance.png', dpi=300, bbox_inches='tight')
plt.show()
```



```
In [63]: import matplotlib.pyplot as plt
import seaborn as sns

# Set the visual style
sns.set(style="whitegrid")
plt.figure(figsize=(10, 8))

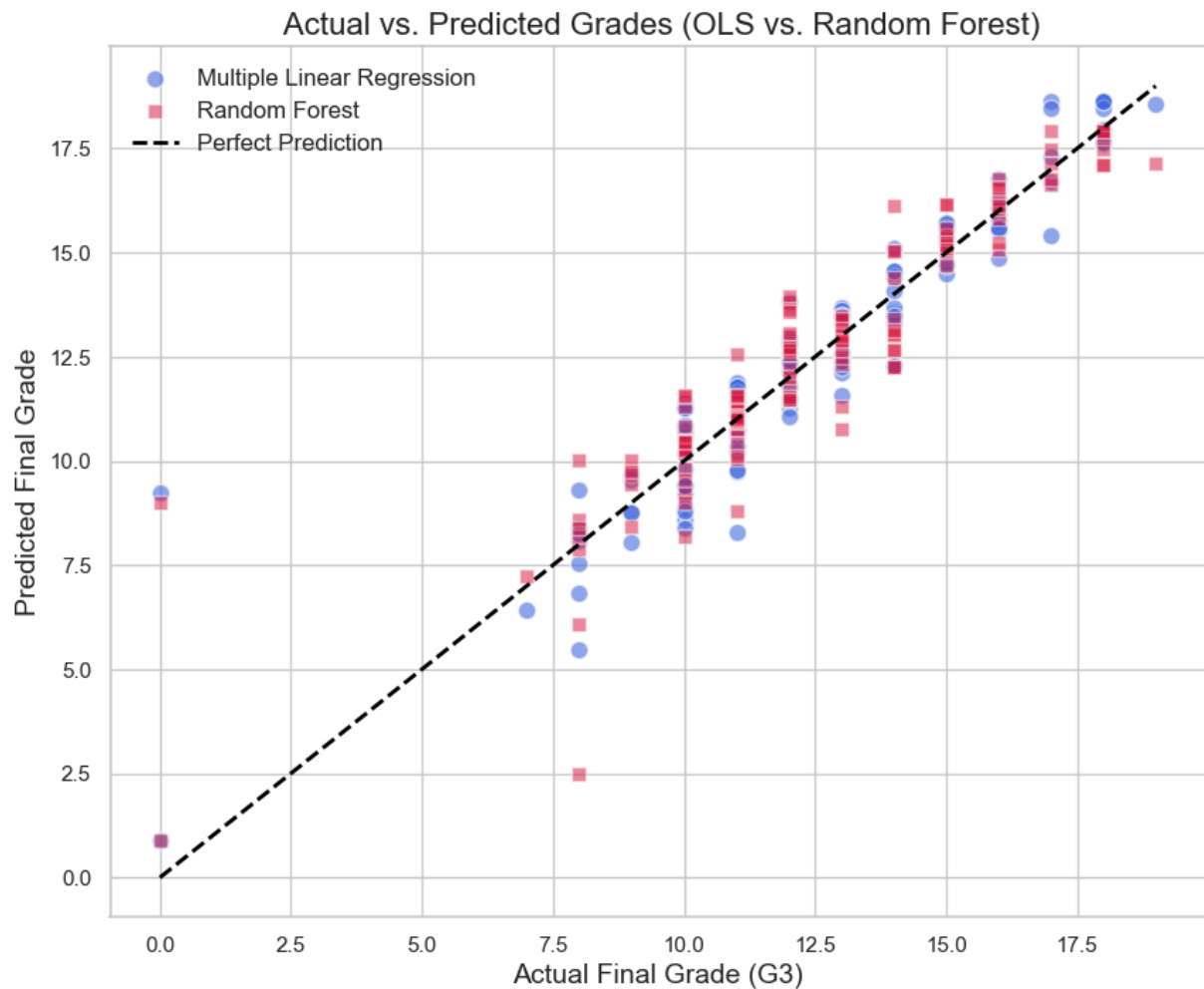
# 1. Plot Multiple Linear Regression results
plt.scatter(y_test, ols_pred, color='royalblue', alpha=0.6, label='Multiple Linear')

# 2. Plot Random Forest results
plt.scatter(y_test, rf_pred, color='crimson', alpha=0.5, label='Random Forest', mar

# 3. Add the identity Line (Perfect Prediction)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='black',
```

```
# 4. Add Labels, title, and Legend
plt.title(' Actual vs. Predicted Grades (OLS vs. Random Forest)', fontsize=16)
plt.xlabel('Actual Final Grade (G3)', fontsize=14)
plt.ylabel('Predicted Final Grade', fontsize=14)
plt.legend(fontsize=12)

# 5. Save and show
plt.savefig('predicted_vs_actual_comparison.png', dpi=300, bbox_inches='tight')
plt.show()
```



In []: